Module 4 Lab 1: pandas on a Real Dataset

20 points. Estimated time 100 to 130 minutes.

Your name: Shane McQueen

Date: 11/10/26

How to use this file. Type your answers straight into the shaded boxes. Do not delete the questions. Anything marked as written in advance has to be filled in before you run that step, not afterwards. When you are finished, save this file and upload it with the rest of your submission.

1. Your unique values

These are what make the submission yours. They come from your own account and your own run, so no two students have the same ones.

The number of distinct data quality problems you found:
2

Your decision and stated cost for the missing unit_price values:

The one problem you found that is not in the troubleshooting table:
21 unit_price values are missing

2. Predictions, written in advance

Write each prediction BEFORE you run that step. A wrong prediction reasoned honestly earns full credit. One edited afterwards to match the result earns none.

Step 1

Before you print anything else, predict: how many distinct data quality problems do you think are in this file? Write a number and, if you can, name one problem you expect to find based on the dtypes alone.

3. Probably more than the two we are asked to find for some variability. Maybe one is the wrong type, like a numeric column is alphanumeric

What actually happened

Dtypes looked correct, descriptive stats showed negative shipping costs and isna() showed missing unit prices

Why it was different, or why I was right

There may be still incorrect entries, but all the dtypes seemed fine. The problems were found in descriptive stats and isna() values

Step 5

Before you decide how to handle the missing unit_price values, predict: will dropping those rows change the mean of any other column by more than one percent?

yes

What actually happened

No the change is less than one percent

Why it was different, or why I was right

It is only 21 out of 500 values, and they must have a similar mean

3. Break it on purpose

Working code shows you what happens. Broken code shows you why. Do each of these deliberately, paste the real error text, and undo the change before moving on.

Break 1. Compute the mean of shipping_cost before removing the negative values, then after.

Paste the exact error text here, not a description of it

```
mean  before   12.209228
mean  after   12.877039
```

Both numbers and the percentage difference. Note how a small number of impossible values moved the summary statistic.

(12.877039-12.209228)/12.209228 X100% = ~5%

Break 2. Fill the missing unit_price values with zero instead of your chosen approach, then recompute average order value by category.

Paste the exact error text here, not a description of it

```
Home  average zeroes =  54.7572101210121
 Home  average original =  56.43345124716552

Toys  average zeroes =  58.74165585519922
 Toys  average original =  59.96544035218253

Apparel  average zeroes =  48.351938880706925
 Apparel  average original =  52.11264523809524

Electronics  average zeroes =  49.567896577380964
 Electronics  average original =  53.466495184590684

Grocery  average zeroes =  67.45879248120302
 Grocery  average original =  68.90951920122889
```

How the category ranking changes, and one sentence on why filling with zero is almost never right for a price.

The category ranking did not change. Filling with zero would greatly bring down other important metrics like cost/profit and make it seem as if they are much lower than actual.

Break 3. Convert the order_date column to datetime with pd.to_datetime and errors='raise'.

Paste the exact error text here, not a description of it

```
0    2026-04-12
1    2026-03-15
2    2026-10-21
3    2026-03-15
4    2026-03-08
       ...
495  2026-07-05
496  2026-03-11
497  2026-10-10
498  2026-09-15
499  2026-06-01
Name: order_date, Length: 466, dtype: datetime64[ns]
```

Whether it succeeds or the exact error. If it succeeds, note why the column loaded as object anyway.

It succeeds. Datetime instances are objects in python

4. Results

The numbers and outputs your own run produced.

Your column by column profile notes

```
       quantity  unit_price  shipping_cost
count  479.000000  479.000000    479.000000
mean     5.079332  189.523591     12.209228
std      2.646141  115.619862      8.368581
min      1.000000    5.070000    -22.560000
25%      3.000000   87.805000      5.510000
50%      5.000000  177.790000     12.610000
75%      7.500000  293.770000     19.115000
max      9.000000  398.810000     24.980000
       quantity  unit_price  shipping_cost
count  466.000000  466.000000    466.000000
mean     5.070815  189.180773     12.877039
std      2.643175  115.187110      7.350172
min      1.000000    5.070000      0.000000
25%      3.000000   88.072500      6.032500
50%      5.000000  177.380000     13.060000
75%      7.000000  288.702500     19.180000
max      9.000000  398.810000     24.980000
(466, 8)
order_id         object
order_date       object
category         object
region           object
quantity          int64
unit_price      float64
shipping_cost     float64
payment_method     object
dtype: object
order_id        0
order_date      0
category        0
region          0
quantity        0
unit_price      0
shipping_cost    0
payment_method    0
dtype: int64
```

Your three decision blocks with stated costs

```
WHAT:   21 unit_price values are missing
DID:    removed 21 improperly reported rows and saved an index of removed values
COSTS:  Lose 21 rows, the mean quantity changed less than 1%

WHAT:   13 shipping_cost values are negative
DID:    removed 13 improperly reported rows and saved an index of removed values
COSTS:  I lose 13 rows in the data, if shipping costs are the correct values but negative I lose valuable data

WHAT:   21 unit_price values are missing
DID:    removed 21 improperly reported rows and saved an index of removed values
COSTS:  Lose 21 rows, the mean quantity changed less than 1%
```

Your three grouped aggregation answers

```
print(df_clean.groupby('category')['quantity'].mean().sort_values(ascending=False))
category
Toys         5.312500      ← Highest average quantity category
Electronics  5.213483
Home         5.183673
Apparel      4.966667
Grocery      4.666667
Name: quantity, dtype: float64

print(df_clean.groupby('region')['shipping_cost'].mean().sort_values(ascending=False))
region
South     13.307869      ← Highest average shipping region
West      13.027647
Central   12.885648
East      12.266667
Name: shipping_cost, dtype: float64
```

5. Reflection

Your own words. No AI assistance on this part. Graders check these answers against the rest of your submission.

Question 1

Report your Step 1 prediction for the number of problems and the number you actually found. If you found one the troubleshooting table does not mention, describe it, because that one is worth the most.

I only found two problems but I had predicted 3.

Question 2

Report your Step 5 prediction about whether dropping rows shifts other columns, and what actually happened with your own numbers. Say what that tells you about whether the missingness is random.

I predicted it would affect the means but it did not shift more than one percent because I removed only 21 out of 500 values, and they must have a similar mean to the rest of the data

Question 3

Pick your most consequential cleaning decision and argue the opposite case. Make the strongest argument you can for the choice you rejected, then say whether writing it changed your mind.

I rejected replacing negative shipping costs with their absolute values because I figured if the numbers were incorrectly entered than the whole row must be counted as unreliable information. Writing this does not change my mind

Question 4

Your second break it exercise filled prices with zero. Describe concretely how a model trained on that version could look fine in testing and produce a specific wrong business outcome in production.

A model trained on this data would have an altered regression line accounting for the false costs, and would assume profitability is lower on average.

Question 5

Someone will train a model on your cleaned file without reading your notebook. Write the three sentence note you would put at the top of the file so they do not misuse it.

This data is not to be used by automated agents or used out of context. This data is only for use within the SMC course cs79f. The data was cleaned by removing NA values and incorrectly entered shipping costs on 11/10/2026.

6. AI use disclosure

Required by the course AI Use Policy.

Tool used (write none if you used no AI):

If you used one: what you asked it, what you kept, and how you checked it

None

7. Before you submit

- ☐ I profiled before I cleaned anything
- ☐ Every decision block has a COST line
- ☐ My reflection quotes numbers from my own run
- ☐ Every prediction was written before I ran that step
- ☐ I recorded the actual result for every prediction, including the wrong ones
- ☐ I pasted the real error text for every break it item
- ☐ My notebook runs top to bottom with every output visible
- ☐ PDF exported after that full run, and it matches
- ☐ Work pushed to my GitHub portfolio repository and the URL is in here
- ☐ Reflection written by me with no AI assistance
- ☐ Files uploaded individually, no archive